

Strings are immutable in java

🤔 Why Are Strings Immutable in Java?

Here are the key reasons:

🔒 1. Security

Strings are commonly used to represent sensitive data like file paths, URLs, class names, and network connections. Making them immutable ensures their content can't be altered after creation, preventing serious vulnerabilities.

📦 2. String Pooling and Memory Efficiency

Java uses a **String pool** to store string literals efficiently. If strings were mutable, then one change would affect all references to that string, which would be dangerous and wasteful.

```
1 String a = "hello";
2 String b = "hello";
3 System.out.println(a == b); // true (same object in the pool)
```

If `a` were mutable and changed, `b` would also change — not acceptable. Immutability solves this.

🔄 3. Thread Safety

Immutable objects are **inherently thread-safe**. Since strings can't be changed, multiple threads can use the same string without synchronization.

📦 4. Reliable Hashing

Strings override `hashCode()` and are often used as keys in `HashMap` and `HashSet`. If strings were mutable, changing a string would change its hash code, breaking the collection's behavior.

🔧 How Can We Prove Strings Are Immutable?

Example 1: Modifying a String

```
1 public class Main {
2     public static void main(String[] args) {
3         String original = "abc";
4         String modified = original.concat("def");
5
6         System.out.println("Original: " + original); // abc
7         System.out.println("Modified: " + modified); // abcdef
8     }
9 }
```

Output:

```
1 | Original: abc  
2 | Modified: abcdef
```

The original string remains unchanged. A new string was created.

document by Suyash 🤖